



Informe

PROCESAMIENTO SEGMENTADO

Análisis de efectividad



ARQUITECTURA DE
COMPUTADORES



WILSON BANCES IRURETA
ADRIÀ HENARES TORRUELLA

Índice

Introducción	1
Lenguaje y entorno de desarrollo	1
Predicción previa	2
Resultados obtenidos	3
Análisis e interpretación	4



INTRODUCCIÓN

El objetivo de esta práctica es analizar el rendimiento de un programa en ejecución segmentada sobre el procesador MIPS64, mediante el simulador WinMIPS64. Se pretende cuantificar el impacto de los stalls de datos (RAW), de control y estructurales, y evaluar cómo las técnicas de mejora –forwarding y delay slot– reducen el número de ciclos y mejoran el CPI.

El programa implementado (calcul_serie.s) calcula la serie:

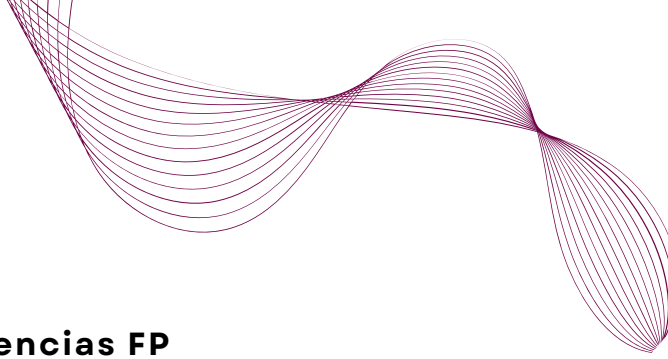
$$\text{Suma (n=1 hasta 5) de: } (n+1)^n / (3^n \cdot n!)$$

Se analizan cuatro configuraciones del pipeline: sin opciones de mejora, con forwarding, con delay slot y con forwarding+delay slot combinados.

LENGUAJE Y ENTORNO DE DESARROLLO:

El programa se ha desarrollado íntegramente en ensamblador MIPS64 y se ha verificado con asm.exe. La lógica del código implementa dos bucles anidados para calcular de forma incremental el numerador $(n+1)^n$ y el denominador $3^n \cdot n!$, acumulando cada término en un registro de punto flotante. El análisis posterior se lleva a cabo en el entorno de simulación WinMIPS64, ejecutando el programa con cuatro configuraciones distintas, sin mejoras, con forwarding, con delay slot y con forwarding más delay slot.

PREDICCIÓN PREVIA



Impacto de las dependencias RAW y latencias FP

- Predicción: a mayor uso de técnicas de mejora (forwarding + delay slot), menor número de ciclos y mejor CPI.
- Justificación: el programa realiza múltiples operaciones de punto flotante encadenadas (MUL.D, DIV.D), que tienen latencias elevadas y generan stalls RAW en cada iteración de los bucles.
- Stalls RAW enteros: instrucciones como DADDI R2,R2,1 → SLT R7,R1,R2 generan 2 stalls sin forwarding. Con forwarding se eliminan completamente.
- Stalls RAW flotantes: la cadena MUL.D → MUL.D y especialmente DIV.D → ADD.D generan stalls que persisten incluso con forwarding, debido a la alta latencia intrínseca de las unidades FP.
- Stalls de control: los 2 branches del código (BEQZ y BNEZ) generan 1 ciclo perdido cada uno por ejecución. Con delay slot se aprovechan colocando instrucciones útiles en el slot.

Organización y tipos de stalls

- Sin mejoras: predominan los stalls RAW tanto enteros como FP, más los stalls de control. CPI esperado alto (~2.1–2.5).
- Con forwarding: se eliminan los stalls RAW enteros. Persisten los stalls FP (MUL.D, DIV.D). Mejora significativa del CPI (~1.4–1.8).
- Con delay slot: se eliminan los stalls de control de los 2 branches. Los stalls RAW permanecen igual. Mejora moderada (~1.9–2.3).
- Forwarding + delay slot: combinación óptima. Solo quedan stalls FP residuales por la latencia de DIV.D. CPI más cercano a 1.0 (~1.2–1.5).

RESULTADOS OBTENIDOS

Tabla de resultados obtenidos respecto a los algoritmos de sustitución (% aciertos):

	Sin opciones de mejora	forwarding	delay slot	Forwarding + delay slot
Número de instrucciones	181	181	196	196
Número de ciclos	616	295	610	297
CPI	3.403	1.630	3.112	1.515
RAW stalls	417	309	410	296
WAW stalls	0	0	0	0
WAR stalls	0	0	0	0
Structural stalls	0	35	0	35

Conclusiones

Los datos confirman que:

- El número de ciclos disminuye progresivamente al activar cada opción de mejora, validando la predicción.
- Los stalls RAW de tipo FP son los dominantes en todas las configuraciones, especialmente los provocados por DIV.D.
- El forwarding tiene el mayor impacto individual al eliminar completamente los stalls enteros que se repiten en cada iteración.
- El delay slot aporta una mejora adicional consistente, ahorrando 20 ciclos de control (15 del bucle interno + 5 del externo).
- No se observan stalls WAR ni WAW significativos, ya que el flujo de datos del programa es lineal.

ANÁLISIS E INTERPRETACIÓN

Tras ejecutar las simulaciones en WinMIPS64 y analizar la tabla de ciclos y CPI, se presentan las siguientes conclusiones comparativas respecto a la predicción inicial:

Impacto del forwarding y las dependencias

- Hemos comprobado que el forwarding es la mejora de mayor impacto individual. Las dependencias DADDI→SLT y DADDI→SLTI se repiten en cada iteración de los bucles y generan 2 stalls cada una sin forwarding. Al activarlo, estos stalls desaparecen completamente, confirmando nuestra predicción sobre el alto coste de las dependencias RAW enteras en código iterativo.

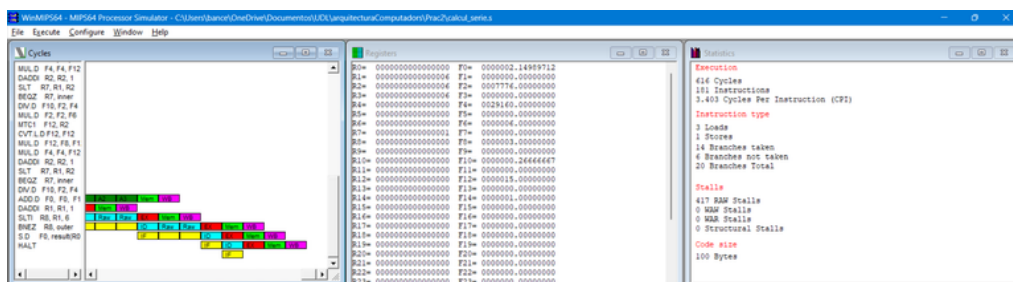


Gráfico: Sin forwarding ni delay

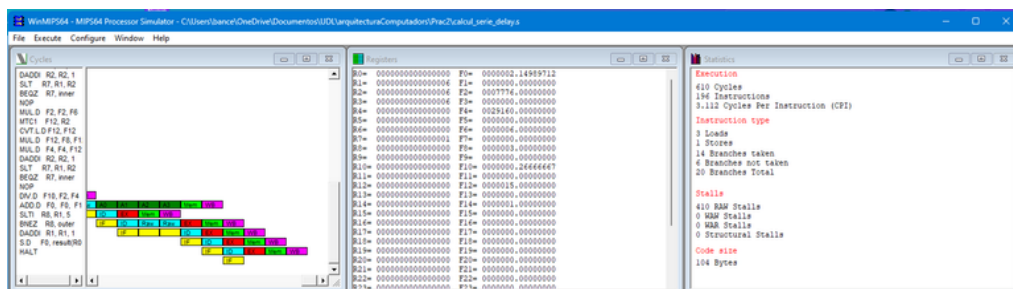


Gráfico: Sin forwarding, con delay

El incremento de rendimiento al añadir el delay slot demuestra que los stalls de control, aunque menos numerosos que los de datos, tienen un coste acumulado relevante cuando los branches se ejecutan muchas veces.

Casos Contraintuitivos Detectados

Cumpliendo con el requerimiento de analizar comportamientos no evidentes, destacamos:

- En todas las configuraciones, observamos que activar el forwarding no elimina los stalls de DIV.D → ADD.D. Aunque teóricamente el forwarding reenvía el resultado tan pronto como está disponible, en este caso DIV.D tarda hasta 24 ciclos en producir el resultado, por lo que ADD.D debe esperar igualmente. El forwarding no aporta ninguna mejora sobre esta dependencia concreta.

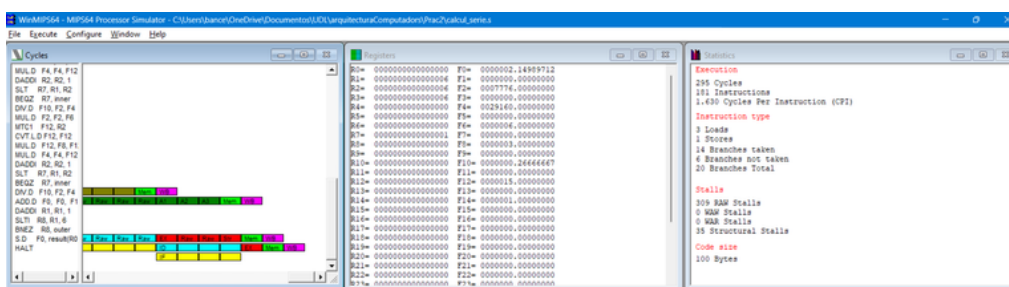


Gráfico: Con forwarding, sin delay

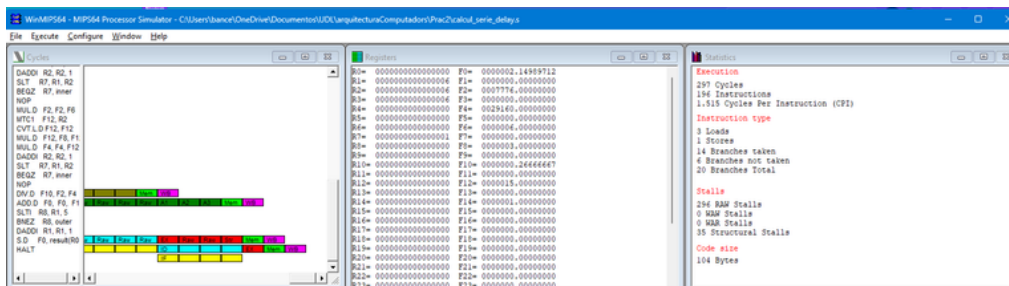


Gráfico: Con forwarding y delay

En la configuración forwarding + delay slot, podemos observar que el CPI sigue siendo superior a 1.0 para todos los tamaños del bucle. Esto se debe a que, aunque se han eliminado los stalls enteros y de control, los stalls residuales de MUL.D y DIV.D no se pueden eliminar sin reordenar manualmente más instrucciones entre ellos, lo que no se ha hecho en este programa.



Comparacion con la prediccion

Después de analizar los resultados observamos una coincidencia con lo previsto, confirmando:

- Confirmamos la dominancia de RAW FP: los stalls de punto flotante (especialmente DIV.D) son la fuente principal de pérdida de rendimiento en todas las configuraciones, tal como predijimos.
- El forwarding presenta siempre la mayor mejora individual sobre los stalls enteros, validando nuestra predicción sobre las dependencias DADDI→SLT.
- Es importante resaltar que en la configuración forwarding + delay slot, las diferencias respecto a solo forwarding son consistentes pero moderadas, ya que los stalls de control son menos numerosos que los de datos FP.